

SmartRFP — an AI Document Processing System built on RAG + Live Web Search.

This document explains the architecture end to end: how a document moves from upload to a finalized, human-approved output, which two agents do the heavy lifting, the technology behind each layer, real-time scenarios, UI mockups, and test cases.

UI: Streamlit

Backend: FastAPI

LLM: OpenAI GPT-4 / 3.5

Orchestration: LangChain

Vector DB: pgvector / FAISS

Web: SerpAPI / Tavily

DB: PostgreSQL

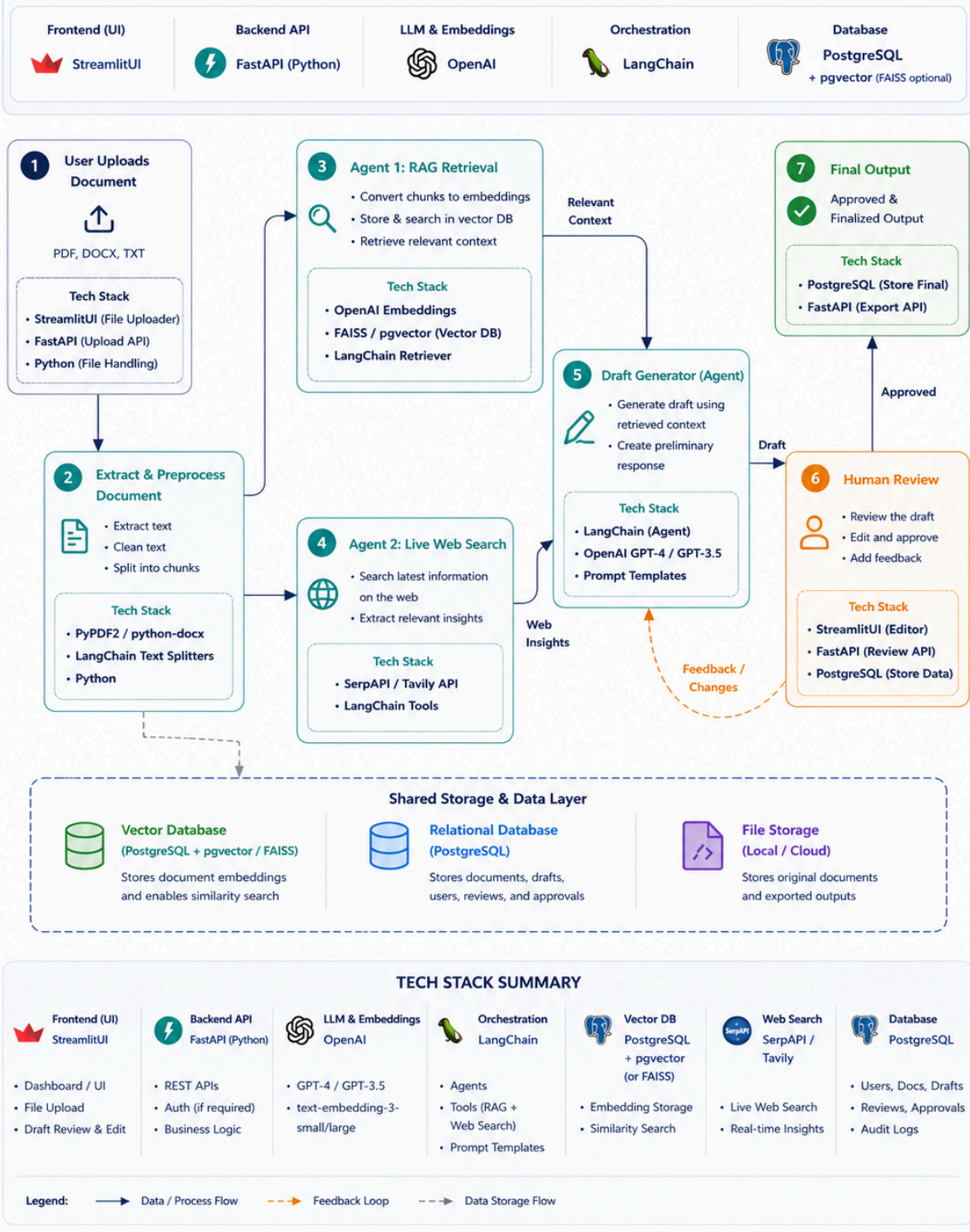
SECTION 01

The system at a glance

The pipeline takes an uploaded document, understands it, gathers knowledge from **two sources at once** — the company's own indexed documents (Agent 1) and the live web (Agent 2) — and writes a draft. A human reviews and approves it before it becomes final. LangChain coordinates the agents and tools; everything is stored in a shared data layer.

AI Document Processing System

Automated document analysis, drafting, and review using RAG + LLMs



THE FULL ARCHITECTURE — NUMBERS 1-7 ARE THE PROCESS FLOW; THE DATA LAYER SITS UNDERNEATH

SECTION 02

End to end — every step

Follow the numbers from the diagram. Each step shows what goes in, what comes out, and the technology used.

1

User uploads document entry point

The user uploads a PDF, DOCX, or TXT through the Streamlit interface. FastAPI receives the file and starts a new processing job.

OUT: stored raw file **TECH:** Streamlit uploader · FastAPI upload API · Python

2

Extract & preprocess automated

The text is extracted from the file, cleaned (headers, noise removed), and split into smaller **chunks**. Chunking matters: small, focused passages retrieve far more accurately than whole pages.

IN: raw file **OUT:** clean text chunks

TECH: PyPDF2 / python-docx · LangChain text splitters

3

Agent 1 — RAG Retrieval runs in parallel

Chunks are converted to embeddings and stored in the vector database. For each question, the agent searches the vector DB and returns the most relevant internal context. *(Detailed in Section 03.)*

IN: chunks + vector DB **OUT:** relevant internal context

TECH: OpenAI embeddings · FAISS/pgvector · LangChain retriever

4

Agent 2 — Live Web Search runs in parallel

At the same time, this agent queries the live web for the latest information the internal documents may not have, and extracts the useful insights. *(Detailed in Section 03.)*

IN: query topics **OUT:** fresh web insights **TECH:** SerpAPI / Tavily · LangChain tools

5

Draft Generator (Agent)

synthesis

This agent combines the **relevant internal context** (from Agent 1) and the **web insights** (from Agent 2) and generates a preliminary draft using prompt templates and the LLM.

IN: internal context + web insights **OUT:** draft response

TECH: LangChain agent · GPT-4 / 3.5 · prompt templates

6

Human Review

human-in-the-loop

A reviewer reads the draft, edits it, and either approves it or adds feedback. If they request changes, a **feedback loop** sends their notes back to the Draft Generator (step 5) to regenerate — shown as the orange dashed line in the diagram.

IN: draft **OUT:** approve → step 7 · or feedback → step 5

TECH: Streamlit editor · FastAPI review API · PostgreSQL

7

Final Output

exit point

The approved, finalized document is stored and made available for export. The relational database keeps the record; file storage keeps the exported file.

IN: approved draft **OUT:** finalized document + stored record

TECH: PostgreSQL (store final) · FastAPI export API

Why two agents? Agent 1 answers from what the company *already knows* (accurate, grounded, private). Agent 2 fills the gaps with what's *new on the web* (current). Combining both gives a draft that is both grounded and up to date.

SECTION 03

The agents — what each one does

The system uses two retrieval agents plus a generator agent. Here is how each works internally.

Agent 1 · RAG Retrieval

INTERNAL KNOWLEDGE — GROUNDED IN YOUR OWN DOCUMENTS

- 1 Embed the chunks.** Each text chunk from step 2 is turned into a vector using OpenAI embeddings (e.g. text-embedding-3-small), capturing its meaning as numbers.
- 2 Store in the vector DB.** Vectors are saved in pgvector (or FAISS) so they can be searched by similarity.
- 3 Search by meaning.** For each question, the question is embedded and compared to stored vectors; the closest chunks are returned — even when wording differs.
- 4 Return relevant context.** The top chunks become the grounded "evidence" the Draft Generator will write from, each tied to its source document.

Used for: matching the document's requirements against existing internal knowledge — past responses, templates, solution docs, policies, and reference material.

Agent 2 · Live Web Search

EXTERNAL KNOWLEDGE — CURRENT INFORMATION FROM THE WEB

- 1 Decide what to look up.** From the document's topics, the agent forms targeted web queries — typically things that change over time or aren't in internal docs.
- 2 Search the web.** It calls a search API (SerpAPI or Tavily) through LangChain tools to get the latest results.
- 3 Extract insights.** It reads the top results and pulls out the relevant facts, figures, or updates.
- 4 Return web insights.** These insights, with their links, are handed to the Draft Generator alongside the internal context.

Used for: latest standards, regulations, market data, competitor or product updates — anything the internal library wouldn't contain yet.

Draft Generator · Agent

SYNTHESIS — TURNS EVIDENCE INTO A WRITTEN DRAFT

- 1 Gather both inputs.** It receives the internal context (Agent 1) and the web insights (Agent 2).
- 2 Apply prompt templates.** Structured prompts tell the LLM how to format the response and in what tone.
- 3 Generate the draft.** GPT-4 / 3.5 writes a preliminary response grounded in the supplied evidence, keeping sources attached.
- 4 Hand off for review.** The draft moves to Human Review; if feedback comes back, this agent regenerates the affected parts.

SECTION 04

Shared storage & data layer

Three stores sit beneath the whole pipeline, shown at the bottom of the diagram.

■ Vector Database

pgvector / FAISS. Stores document embeddings and powers similarity search for Agent 1.

■ Relational Database

PostgreSQL. Stores documents, drafts, users, reviews, approvals, and audit logs.

■ File Storage

Local / Cloud. Keeps the original uploaded files and the exported final outputs.

SECTION 05

Real-time scenarios

How the system behaves in practice — including the feedback loop and the value of live web search.

SCENARIO A • THE HAPPY PATH

From upload to approved output in one pass

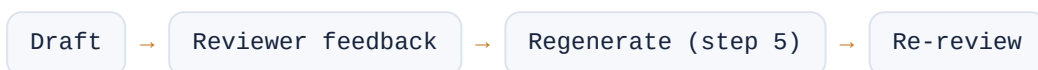
A user uploads a 30-page document. It is chunked and embedded. Agent 1 retrieves the matching internal context while Agent 2 pulls two recent web updates. The Draft Generator produces a complete draft with sources. The reviewer reads it, makes one small edit, and approves. The final output is stored and exported.



SCENARIO B • THE FEEDBACK LOOP

Reviewer requests changes; the draft is regenerated

The reviewer finds section 4 too generic and adds feedback: "expand with the latest figures." The system sends that note back to the Draft Generator (the orange dashed line). The agent regenerates that section — re-querying Agent 2 for fresher numbers — and returns it for a second review. Nothing is finalized until the reviewer approves.



SCENARIO C • LIVE WEB SEARCH ADDS VALUE

The document asks about something newer than the internal library

The document references a standard updated last month. Agent 1 finds the old version internally; Agent 2 searches the web and retrieves the latest revision. The Draft Generator uses the current version and cites the web source, so the response is accurate and defensible.



SECTION 06

UI mockup (Streamlit)

Two key screens in the Streamlit app: the upload screen and the human review & edit screen.

Screen 1 · Upload & process

 SmartRFP

 Upload

 Drafts

 Review

 Knowledge base

 Settings

Upload a document

Supported: PDF, DOCX, TXT

Drag and drop file here

Limit 50MB per file

Browse files

DOCUMENT TITLE

Acme Cloud Migration — proposal draft

USE LIVE WEB SEARCH?

✓ Yes — fetch latest information

Process document

Save

Screen 2 · Human review & edit

 SmartRFP

 Upload

 Drafts

 Review

 Knowledge base

 Settings

Review draft — Acme Cloud Migration

Generated from 38 internal chunks + 2 web sources

Solution Overview

We propose a phased migration with a managed platform and 24/7 support. `src: past-proposal-2025`

Compliance — Latest Standard

Aligned to the revised data-protection guidance issued this quarter. `web: regulator.gov`

Add feedback: "Expand section 4 with the latest cost figures." — sends back to the Draft Generator to regenerate.



✓ Approve & finalize

 Edit inline

↩ Send feedback

SECTION 07

Test cases

Coverage across the pipeline, the agents, the human-in-the-loop feedback, and failure cases.

 Functional  Flow / integration  Human-in-the-loop  Edge / failure

ID	TYPE	TEST	EXPECTED RESULT
TC-01	Func	Upload a valid PDF, DOCX, and TXT.	Each file accepted and stored; a processing job starts.
TC-02	Func	Run extract & preprocess on a multi-section file.	Text extracted and cleaned; split into chunks with no content lost.
TC-03	Func	Embed chunks and store in the vector DB.	Embeddings created and saved; count matches number of chunks.
TC-04	Func	Agent 1 retrieves context for a known query.	Most relevant chunks returned, ranked by similarity, with sources.
TC-05	Func	Agent 2 searches the web for a current topic.	Fresh results returned; key insights extracted with links.
TC-06	Flow	Draft Generator combines internal context + web insights.	Coherent draft produced using both sources, with citations.
TC-07	Flow	Run the full pipeline end to end on one document.	Upload → extract → both agents → draft → review → final, with data saved at each stage.
TC-08	HITL	Reviewer edits the draft and approves.	Edits saved; status set to Approved; final output stored.
TC-09	HITL	Reviewer adds feedback requesting changes.	Feedback routed to Draft Generator; section regenerated; returned for re-review.
TC-10	HITL	Attempt to export without approval.	Export blocked; nothing finalized without human approval.
TC-11	Edge	Web search API is unavailable.	Pipeline continues with internal context only; missing web data flagged, not faked.
TC-12	Edge	Vector DB returns no relevant context.	Section marked "no internal match"; reviewer alerted rather than a guessed answer.

TC-13	Edge	Corrupted or password-locked / oversized file uploaded.	Clear error message; user prompted to re-upload; no silent failure.
TC-14	Edge	Very large document (hundreds of pages).	Chunking and retrieval still complete; performance stays within limits.

SECTION 08

Technology stack summary

LAYER	TECHNOLOGY	ROLE IN THE SYSTEM
Frontend (UI)	Streamlit	Dashboard, file upload, draft review & inline editing.
Backend API	FastAPI (Python)	REST APIs, auth (if needed), business logic, runs the pipeline.
LLM & Embeddings	OpenAI	GPT-4 / 3.5 for drafting; text-embedding-3 for vectorizing text.
Orchestration	LangChain	Agents, tools (RAG + web search), prompt templates, retrievers.
Vector DB	PostgreSQL + pgvector (or FAISS)	Embedding storage and similarity search for Agent 1.
Web Search	SerpAPI / Tavily	Live web search and real-time insights for Agent 2.
Database	PostgreSQL	Users, documents, drafts, reviews, approvals, audit logs.
File Storage	Local / Cloud	Original uploads and exported final documents.

Presenting tip: the headline is "two knowledge sources, one grounded draft, one human gate." Agent 1 = what we already know (private + accurate). Agent 2 = what's new (web + current). The human review + feedback loop is what keeps quality high.

SmartRFP · AI Document Processing System — Streamlit · FastAPI · OpenAI · LangChain · pgvector/FAISS ·
SerpAPI/Tavily · PostgreSQL